pantable

Release 0.14.2

Kolen Cheung

Jan 26, 2022

CONTENTS:

CHAPTER

ONE

PANTABLE—A PYTHON LIBRARY FOR WRITING PANDOC FILTERS FOR TABLES WITH BATTERIES INCLUDED.

Date January 25, 2022

Contents

1.1 Introduction

Pantable is a Python library that maps the pandoc Table AST to an internal structure losslessly. This enables writing pandoc filters specifically manipulating tables in pandoc.

This also comes with 3 pandoc filters, `pantable`, `pantable2csv`, `pantable2csvx`.

`pantable` is the main filter, introducing a syntax to include CSV table in markdown source. It supports all table features supported by the pandoc Table AST.

`pantable2csv` complements `pantable`, is the inverse of `pantable`, which convert native pandoc tables into the CSV table format defined by `pantable`. This is lossy as of pandoc 2.11+, which is supported since pantable 0.13.

`pantable2csvx` (experimental, may drop in the future) is similar to `pantable2csv`, but introduces an extra column with the `fancy-table` syntax defined below such that any general pandoc Table AST can be losslessly encoded in CSV format.

Some example uses are:

1. You already have tables in CSV format.
2. You feel that directly editing markdown table is troublesome. You want a spreadsheet interface to edit, but want to convert it to native pandoc table for higher readability. And this process might go back and forth.
3. You want lower-level control on the table and column widths.
4. You want to use all table features supported by the pandoc's internal AST table format, which is not possible in markdown for pandoc (as of writing.)

1.1.1 A word on support

Note that the above is exactly how I use pantable personally. So you can count on the round-trip losslessness. `pantable` and `pantable2csv` should have robust support since it has been used for years. But since pandoc 2.11 the table AST has been majorly revised. Pantable 0.13 added support for this new AST by completely rewriting pantable, at the same time addresses some of the shortcoming of the original design. Part of the new design is to enable pantable as a library (see *Pantable as a library* below) so that its functionality can be extended, similar to how to write a pandoc filter to intercept the AST and modify it, you can intercept the internal structure of PanTable and modify it.

However, since this library is completely rewritten as of v0.13,

- `pantable` and `pantable2csv` as pandoc filters should be stable
  - there may be regression, please open an issue to report this
- round-trip losslessness may break, please open an issue to report this
- `pantable2csvx` as pandoc filter is experimental. API here might change in the future or may be dropped completed (e.g. replaces by something even more general)
- Pantable as a library also is experimental, meaning that the API might be changed in the future.

1.2 Installation

1.2.1 Pip

To manage pantable using pip, open the command line and run

- `pip install pantable` to install
  - `pip install https://github.com/ickc/pantable/archive/master.zip` to install the in-development version
- `pip install -U pantable` to upgrade
- `pip uninstall pantable` to remove

You need a matching pandoc version for pantable to work flawlessly. See *Supported pandoc versions* for details. Or, use the *Conda* method to install below to have the pandoc version automatically managed for you.

1.2.2 Conda

To manage pantable **with a matching pandoc version**, open the command line and run

- `conda install -c conda-forge pantable` to install
- `conda update pantable` to upgrade
- `conda remove pantable` to remove

You may also replace `conda` by `mamba`, which is basically a drop-in replacement of the conda package manager. See mamba-org/mamba: The Fast Cross-Platform Package Manager for details.

1.2.3 Note on versions

Supported Python versions

pantable v0.12 drop Python 2 support. You need to `pip install pantable<0.12` if you need to run it on Python 2.

To enforce using Python 3, depending on your system, you may need to specify `python3` and `pip3` explicitly.

Check the badge above or `setup.py` for supported Python versions, `setup.py` further indicates support of pypy in additional of CPython.

Supported pandoc versions

pandoc versioning semantics is MAJOR.MAJOR.MINOR.PATCH and panflute's is MAJOR.MINOR.PATCH. Below we shows matching versions of pandoc that panflute supports, in descending order. Only major version is shown as long as the minor versions doesn't matter.

Table 1: Version Matching[Page 4, 1]

pantable	panflute version	supported pandoc versions	supported pandoc API versions
0.14.1	2.1.3	2.11.0.4–2.16.x	1.22–1.22.1
0.14	2.1	2.11.0.4—2.14.x	1.22
0.13	2.0	2.11.0.4—2.11.x	1.22
•	not supported	2.10	1.21
0.12	1.12	2.7-2.9	1.17.5–1.20

Note: pandoc 2.10 is short lived and 2.11 has minor API changes comparing to that, mainly for fixing its shortcomings. Please avoid using pandoc 2.10.

To use pantable with pandoc < 2.10, install pantable 0.12 explicitly by `pip install pantable~=0.12.4`.

1.3 Pantable as pandoc filters

1.3.1 `pantable`

This allows CSV tables, optionally containing markdown syntax (disabled by default), to be put in markdown as a fenced code blocks.

1.3.2 Example

Also see the README in GitHub Pages.

````
```table
---
caption: '*Awesome* **Markdown** Table'
alignment: RC
table-width: 2/3
markdown: True
---
First row,defaulted to be header row,can be disabled
1,cell can contain **markdown**,"It can be aribrary block element:

- following standard markdown syntax
- like this"
2,"Any markdown syntax, e.g.",E = mc^2^
```
````

becomes

[1] For pandoc API verion, check https://hackage.haskell.org/package/pandoc for pandoc-types, which is the same thing.

Table 2: *Awesome* **Markdown** Table

First row	defaulted to be header row	can be disabled
1	cell can contain **markdown**	It can be aribrary block element: • following standard markdown syntax • like this
2	Any markdown syntax, e.g.	$E = mc^2$

(The equation might not work if you view this on PyPI.)

1.3.3 Usage

```
pandoc -F pantable -o README.html README.md
```

1.3.4 Syntax

Fenced code blocks is used, with a class `table`. See *Example*.

Optionally, YAML metadata block can be used within the fenced code block, following standard pandoc YAML metadata block syntax. 7 metadata keys are recognized:

`caption` the caption of the table. Can be block-like. If omitted, no caption will be inserted. Interpreted as markdown only if `markdown: true` below.

Default: disabled.

`short-caption` the short-caption of the table. Must be inline-like element. Interpreted as markdown only if `markdown: true` below.

Default: disabled.

`alignment` alignment for columns: a string of characters among `L,R,C,D`, case-insensitive, corresponds to Left-aligned, Right-aligned, Center-aligned, Default-aligned respectively. e.g. `LCRD` for a table with 4 columns.

You can specify only the beginning that's non-default. e.g. `DLCR` for a table with 8 columns is equivalent to `DLCRDDDD`.

Default: `DDD...`

`alignment-cells` alignment per cell. One row per line. A string of characters among `L,R,C,D`, case-insensitive, corresponds to Left-aligned, Right-aligned, Center-aligned, Default-aligned respectively. e.g.

```
LCRD
DRCL
```

for a table with 4 columns, 2 rows.

you can specify only the top left block that is not default, and the rest of the cells with be default to default automatically. e.g.

```
DC
LR
```

for a table with 4 columns, 3 rows will be equivalent to

```
DCDD
LRDD
DDDD
```

Default: `DDD...\n...`

width a list of relative width corresponding to the width of each columns. `D` means default width. e.g.

```
- width
    - 0.1
    - 0.2
    - 0.3
    - 0.4
    - D
```

Again, you can specify only the left ones that are non-default and it will be padded with defaults.

Default: `[D, D, D, ...]`

table-width the relative width of the table (e.g. relative to `\linewidth`). If specified as a number, and if any of the column width in `width` is default, then auto-width will be performed such that the sum of `width` equals this number.

Default: None

header If it has a header row or not.

Default: True

markdown If CSV table cell contains markdown syntax or not.

Default: False

fancy_table if true, then the first column of the table will be interpreted as a special fancy-table syntax s.t. it encodes which rows are

- table-header,
- table-foot,
- multiple table-bodies and
- "body-head" within table-bodies.

see example below.

include the path to an CSV file, can be relative/absolute. If non-empty, override the CSV in the CodeBlock.

Default: None

include-encoding if specified, the file from `include` will be decoded according to this encoding, else assumed to be UTF-8. Hint: if you save the CSV file via Microsoft Excel, you may need to set this to `utf-8-sig`.

csv-kwargs If specified, should be a dictionary passed to `csv.reader` as options. e.g.

```
---
csv-kwargs:
  dialect: unix
  key: value...
...
```

format The file format from the data in code-block or include if specified.

Default: `csv` for data from code-block, and infer from extension in include.

Currently only `csv` is supported.

ms (experimental, may drop in the future): a list of int that specifies the number of rows per row-block. e.g. `[2, 6, 3, 4, 5, 1]` means the table should have 21 rows, first 2 rows are table-head, last 1 row is table-foot, there are 2 table-bodies (indicated by `6, 3, 4, 5` in the middle) where the 1st body `6, 3` has 6 body-head and 3 "body-body", and the 2nd body `4, 5` has 4 body-head and 5 "body-body".

If this is specified, `header` will be ignored.

Default: None, which would be inferred from `header`.

ns_head (experimental, may drop in the future): a list of int that specifies the number of head columns per table-body. e.g. `[1, 2]` means the 1st table-body has 1 column of head, the 2nd table-body has 2 column of head

Default: None

1.3.5 pantable2csv

This one is the inverse of `pantable`, a panflute filter to convert any native pandoc tables into the CSV table format used by pantable.

Effectively, `pantable` forms a "CSV Reader", and `pantable2csv` forms a "CSV Writer". It allows you to convert back and forth between these 2 formats.

For example, in the markdown source:

```
+--------+---------------------+--------------------------+
| First  | defaulted to be     | can be disabled          |
| row    | header row          |                          |
+========+=====================+==========================+
| 1      | cell can contain    | It can be aribrary block |
|        | **markdown**        | element:                 |
|        |                     |                          |
|        |                     | -   following standard   |
|        |                     |     markdown syntax      |
|        |                     | -   like this            |
+--------+---------------------+--------------------------+
| 2      | Any markdown        | $$E = mc^2$$             |
|        | syntax, e.g.        |                          |
+--------+---------------------+--------------------------+

: *Awesome* **Markdown** Table
```

running `pandoc -F pantable2csv -o output.md input.md`, it becomes

```
``` {.table}
---
alignment: DDD
caption: '*Awesome* **Markdown** Table'
header: true
markdown: true
table-width: 0.8055555555555556
width: [0.125, 0.3055555555555556, 0.375]
```

(continues on next page)
```

(continued from previous page)

```
---
First row,defaulted to be header row,can be disabled
1,cell can contain **markdown**,"It can be aribrary block element:

-   following standard markdown syntax
-   like this
"
2,"Any markdown syntax, e.g.",$$E = mc^2$$
```

1.3.6 pantable2csvx

(experimental, may drop in the future)

Similar to `pantable2csv`, but convert with `fancy_table` syntax s.t. any general Table in pandoc AST is in principle losslessly converted to a markdown-ish syntax in a CSV representation.

e.g.

```
pandoc -F pantable2csvx -o tests/files/native_reference/planets.md tests/files/native/
↪planets.native
```

would turn the native Table from `platnets.native`[2] to

```
``` {.table}
---
caption: Data about the planets of our solar system.
alignment: CCDRRRRRRR
ns-head:
- 3
markdown: true
fancy-table: true
...
===,"(1, 2)
",,Name,Mass (10\^24kg),Diameter (km),Density (kg/m\^3),Gravity (m/s\^2),Length of day␣
↪(hours),Distance from Sun (10\^6km),Mean temperature (C),Number of moons,Notes
,"(4, 2)
Terrestrial planets",,Mercury,0.330,"4,879",5427,3.7,4222.6,57.9,167,0,Closest to the Sun
,,,Venus,4.87,"12,104",5243,8.9,2802.0,108.2,464,0,
,,,Earth,5.97,"12,756",5514,9.8,24.0,149.6,15,1,Our world
,,,Mars,0.642,"6,792",3933,3.7,24.7,227.9,-65,2,The red planet
,"(4, 1)
Jovian planets","(2, 1)
Gas giants",Jupiter,1898,"142,984",1326,23.1,9.9,778.6,-110,67,The largest planet
,,,Saturn,568,"120,536",687,9.0,10.7,1433.5,-140,62,
,,"(2, 1)
Ice giants",Uranus,86.8,"51,118",1271,8.7,17.2,2872.5,-195,27,
,,,Neptune,102,"49,528",1638,11.0,16.1,4495.1,-200,14,
___,"(1, 2)
```

(continues on next page)

[2] copied from pandoc from here, which was dual licensed as CC0 here
```

(continued from previous page)

```
Dwarf planets",,Pluto,0.0146,"2,370",2095,0.7,153.3,5906.4,-225,5,Declassified as a␣
↪planet in 2006.
```
```

## 1.4 Pantable as a library

(experimental, API may change in the future)

Documentation here is sparse, partly because the upstream (pandoc) may change the table AST again. See Crazy ideas: table structure from upstream GitHub.

See the API docs in https://ickc.github.io/pantable/.

For example, looking at the source of `pantable` as a pandoc filter, in `codeblock_to_table.py`, you will see the main function doing the work is now

```
pan_table_str = (
    PanCodeBlock
    .from_yaml_filter(options=options, data=data, element=element, doc=doc)
    .to_pantablestr()
)
if pan_table_str.table_width is not None:
    pan_table_str.auto_width()
return (
    pan_table_str
    .to_pantable()
    .to_panflute_ast()
)
```

You can see another example from `table_to_codeblock.py` which is what `pantable2csv` and `pantable2csvx` called.

Below is a diagram illustrating the API:

Fig. 1: Overview

Solid arrows are lossless conversions. Dashed arrows are lossy.

You can see the pantable internal structure, `PanTable` is one-one correspondence to the pandoc Table AST. Similarly for `PanCodeBlock`.

It can then losslessly converts between PanTable and PanTableMarkdown, where everything in PanTableMarkdown is now markdown strings (whereas those in PanTable are panflute or panflute-like AST objects.)

Lastly, it defines a one-one correspondence to PanCodeBlock with `fancy_table` syntax mentioned earlier.

Below is the same diagram with the method names. You'd probably want to zoom into it to see it clearly.

Fig. 2: Detailed w/ methods
```

1.5 Development

To run all the tests run `tox`. GitHub Actions is used for CI too so if you fork this you can check if your commits passes there.

1.6 Related Filters

(The table here is created in the beginning of pantable, which has since added more features. This is left here for historical reason and also as a credit to those before this.)

The followings are pandoc filters written in Haskell that provide similar functionality. This filter is born after testing with theirs.

- baig/pandoc-csv2table: A Pandoc filter that renders CSV as Pandoc Markdown Tables.
- mb21/pandoc-placetable: Pandoc filter to include CSV data (from file or URL)
- sergiocorreia/panflute/csv-tables.py

	pandoc-csv2table	pandoc-placetable	panflute example	pantable
caption	caption	caption	title	caption
aligns	aligns = LRCD	aligns = LRCD		aligns = LRCD
width		widths = "0.5 0.2 0.3"		width: [0.5, 0.2, 0.3]
table-width				table-width: 1.0
header	header = yes | no	header = yes | no	has_header: True | False	header: True | False | yes | NO
mark-down		inlinemark-down		markdown: True | False | yes | NO
source	source	file	source	include
others	type = simple | multiline | grid | pipe			
		delimiter		
		quotechar		
		id (wrapped by div)		
Notes				width are auto-calculated when width is not specified

CHAPTER

TWO

INSTALLATION

At the command line:

```
pip install pantable
```

CHAPTER

THREE

USAGE

To use pantable in a project:

```
import pantable
```

CHAPTER

FOUR

CONTRIBUTING

Contributions are welcome, and they are greatly appreciated! Every little bit helps, and credit will always be given.

4.1 Bug reports

When reporting a bug please include:

- Your operating system name and version.
- Any details about your local setup that might be helpful in troubleshooting.
- Detailed steps to reproduce the bug.

4.2 Documentation improvements

pantable could always use more documentation, whether as part of the official pantable docs, in docstrings, or even on the web in blog posts, articles, and such.

4.3 Feature requests and feedback

The best way to send feedback is to file an issue at https://github.com/ickc/pantable/issues.

If you are proposing a feature:

- Explain in detail how it would work.
- Keep the scope as narrow as possible, to make it easier to implement.
- Remember that this is a volunteer-driven project, and that code contributions are welcome :)

4.4 Development

To set up *pantable* for local development:

1. Fork pantable (look for the "Fork" button).
2. Clone your fork locally:

```
git clone git@github.com:YOURGITHUBNAME/pantable.git
```

3. Create a branch for local development:

```
git checkout -b name-of-your-bugfix-or-feature
```

   Now you can make your changes locally.

4. When you're done making changes run all the checks and docs builder with tox one command:

```
tox
```

5. Commit your changes and push your branch to GitHub:

```
git add .
git commit -m "Your detailed description of your changes."
git push origin name-of-your-bugfix-or-feature
```

6. Submit a pull request through the GitHub website.

4.4.1 Pull Request Guidelines

If you need some code review or feedback while you're developing the code just make the pull request.

For merging, you should:

1. Include passing tests (run `tox`).
2. Update documentation when there's new API, functionality etc.
3. Add a note to `CHANGELOG.md` about the changes.

4.4.2 Tips

To run a subset of tests:

```
tox -e envname -- pytest -k test_myfeature
```

To run all the test environments in *parallel*:

```
tox -p auto
```

Contents

CHAPTER

FIVE

REVISION HISTORY FOR PANTABLE

- v0.14.2: Support pandoc 2.15–16
  - improve test matrix in GitHub Actions
  - update dependency constraints
- v0.14.1: identical to v0.14.2
- v0.14.0: Support pandoc 2.14
  - close #61
  - requires panflute >= 2.1
  - add `environment.yml` for an example conda environment running pantable
  - remove `pantable.util.PandocVersion` as it is merged upstream in `panflute.tools.PandocVersion`.
- v0.13.6: Fix #57; update dependencies.
- v0.13.5: Fix a division error when all widths are zero.
- v0.13.4: Fix converting from native table that contains footnotes. See #58.
- v0.13.3: packaging change only: allow Python4
- v0.13.2: packaging change only: specified required versions for all dependencies including extras
- v0.13.1: util.py: fix `iter_convert_texts_markdown_to_panflute`
- v0.13:added pandoc 2.11.0.4+ & panflute 2+ support
  - pandoc 2.10 introduces a new table AST. This version provides complete support of all features supported in the pandoc AST. Hence, older pandoc versions are no longer supported. Install `pantable=0.12.4` if you need to use `pandoc<2.10`.
  - deprecated `pipe_tables`, `grid_tables`, `raw_markdown` options in pantable, which were introduced in v0.12. pantable v0.13 has a much better way to process markdown cells that these are no longer needed.
  - slight changes on markdown output which should be functionally identical. Both changes in pandoc and pantable cause this. See commit eadc6fb.
  - add `short-caption`, `alignment-cells`, `fancy_table`, `format`, `ms`, `ns_head`. See docs for details.
- v0.12.4: Require panflute<2 explicitly
  - panflute 2 is released to support pandoc API 1.22. This release ensures that version control is correct when people specify pantable==0.12 in the future.
- v0.12.3: Fixes test and CI; update on supported Python versions

  - migrate from Travis CI to GitHub Actions
  - supported Python versions are now 3.5-3.8, pypy3
  - minor update in README
- v0.12.2: Add `grid_tables`
- v0.12.1: add `include-encoding`, `csv-kwargs`
  - closes #36, #38. See doc for details on this new keys.
- v0.12: Drop Python 2 support; enhance CSV with markdown performance
  - Dropping Python2 support, existing Python 2 users should be fine with pip>=9.0. See https://python3statement.org/practicalities/.
  - add `pipe_tables`, `raw_markdown` options. See README for details. This for example will speed up CSV with markdown cells a lot (trading correctness for speed though.)

CHAPTER

SIX

PANTABLE

6.1 pantable package

6.1.1 Subpackages

pantable.cli package

Submodules

pantable.cli.pantable module

pantable.cli.pantable.**main**(*doc: Doc | None = None*)
a pandoc filter converting csv table in code block

Fenced code block with class table will be parsed using panflute.yaml_filter with the fuction *pantable.codeblock_to_table.codeblock_to_table()*

pantable.cli.pantable2csv module

pantable.cli.pantable2csv.**main**(*doc: Doc | None = None*)
Covert all tables to CSV table format defined in pantable

- in code-block with class table
- metadata in YAML
- table in CSV

pantable.cli.pantable2csvx module

pantable.cli.pantable2csvx.**main**(*doc: Doc | None = None*)
Covert all tables to CSV table format defined in pantable

- in code-block with class table
- metadata in YAML
- table in CSV

6.1.2 Submodules

pantable.ast module

class pantable.ast.**Align**(*aligns: np.ndarray[np.int8]*)

Bases: object

Alignment class

ALIGN: ClassVar = array(['AlignDefault', 'AlignLeft', 'AlignRight', 'AlignCenter'], dtype='<U12')

aligns: np.ndarray[np.int8]

property aligns_char

property aligns_idx: np.ndarray[np.int8]

this is designed such that aligns_text below works

the last % 4 is to gunrantee garbage input still falls inside the idx range of ALIGN

property aligns_string: str

the aligns string used in pantable codeblock

such as LDRC…

property aligns_text: np.ndarray[np.str_]

classmethod default(*shape: Union[Tuple[int], Tuple[int, int]] = (1,)*) → *Align*

classmethod from_aligns_char(*aligns_char: np.ndarray[np.dtype('S1')]*) → *Align*

classmethod from_aligns_string(*alignment: str*) → *pantable.ast.Align*

create Align from aligns_string

used in __repr__

should not be used by data created by users

should satisfies *Align.from_aligns_string(align.aligns_string) == align*

classmethod from_aligns_string_1d(*alignment: str*, *size: int*) → *pantable.ast.Align*

create Align from aligns_string, 1-dimensional

should be used by data created by users

classmethod from_aligns_string_2d(*alignment_cells: str*, *shape: Tuple[int, int]*) → *Align*

create Align from aligns_string, 2-dimensional

should be used by data created by users

classmethod from_aligns_text(*aligns_text: np.ndarray[Optional[np.str_]]*) → *Align*

class pantable.ast.**Ica**(*identifier: str = ''*, *classes: List[str] = <factory>*, *attributes: Dict[str, str] = <factory>*)

Bases: object

a class of identifier, classes, and attributes

attributes: Dict[str, str]

classes: List[str]

classmethod from_panflute_ast(*elem: ListContainer[Block]*) → *Ica*

identifier: str = ''

to_panflute_ast() → ListContainer[Plain]
to panflute AST element

we choose a ListContainer-Plain-Span here as it is simplest to capture the Ica

class pantable.ast.**PanCodeBlock**(*data: str = ''*, *options: pantable.ast.PanTableOption = <factory>*, *ica: pantable.ast.Ica = <factory>*)

Bases: object

A PanTable representation of CodeBlock

it handles the transition between panflute CodeBlock and PanTable

It can convert to and from panflute CodeBlock, and to and from PanTable

there's no *from_panflute_ast* method, as we expect the args in the *__init__* to be from *panflute.yaml_filter* directly.

c.f. *.util.parse_markdown_codeblock* for testing purposes

data: str = ''

classmethod from_data_format(*data: np.ndarray[np.str_]*, *options: Optional[PanTableOption] = None*, *ica: Optional[Ica] = None*) → *PanCodeBlock*
construct from different data formats

TODO: should io be done by PanCodeBLock.to_panflute_ast or other places? seems wrong to be here

but it may actually belongs to here because where else for binary data?

classmethod from_yaml_filter(*data: str = ''*, *options: Optional[dict] = None*, *element: Optional[CodeBlock] = None*, *doc: Optional[Doc] = None*) → *PanCodeBlock*
these args are those passed from within yaml_filter

ica: *pantable.ast.Ica*

options: *pantable.ast.PanTableOption*

static parse_data_markdown(*str_array: np.ndarray[np.str_]*, *fancy_table: bool = False*, *ica_cell_pat=re.compile('^(\\([0-9,]+\\))?({[^{}]*})?$')*, *fancy_table_pat=re.compile('^({[^{}]*})?? ?(---|===|___)? ?({[^{}]*})?$')*) → Tuple[Optional[np.ndarray[np.int64]], Optional[np.ndarray[np.str_]], np.ndarray[np.str_], np.ndarray[np.str_], *TableArray*]
parse markdown in string array

c.f. PanTableMarkdown.to_str_array

parse_options(*shape: Tuple[int, int]*) → Tuple[str, str, *Spec*, *Align*, Optional[np.ndarray[np.int64]], Optional[np.ndarray[np.int64]]]
parsing PanTableOption to whatever PanTableStr.__init__ needed

This is the point where correctness is checked most aggressively. Here we assumed the types are already correct, so we are checking things beyond types such as Optional, shape, positivity, etc.

to_panflute_ast() → panflute.elements.CodeBlock
return a panflute AST representation

TODO: handle differently if include exists and writable need to be able to configure pantable2csv on write location

to_pantablestr() → *pantable.ast.PanTableStr*
parse data and return a PanTableStr

Exceptions might be raised here

c.f. to_pancodeblock

class pantable.ast.**PanTable**(*cells: Union[TableArray, np.ndarray[Union[ListContainer, str]]], caption: ListContainer[Block] = <factory>, icas_rowblock: Optional[np.ndarray[Ica]] = None, icas_row: Optional[np.ndarray[Ica]] = None, icas: Optional[np.ndarray[Ica]] = None, short_caption: Optional[ListContainer[Inline]] = None, ica_table: Ica = <factory>, spec: Optional[Spec] = None, aligns: Optional[Align] = None, ms: Optional[np.ndarray[np.int64]] = None, ns_head: Optional[np.ndarray[np.int64]] = None*)

Bases: *pantable.ast.PanTableAbstract*

a representation of panflute Table

TableArray should have content type as ListContainer although not strictly enforced here

caption: ListContainer[Block]

classmethod from_panflute_ast(*table: panflute.table_elements.Table*) → *pantable.ast.PanTable*

icas: Optional[np.ndarray[Ica]] = None

icas_row: Optional[np.ndarray[Ica]] = None

icas_rowblock: Optional[np.ndarray[Ica]] = None

static iter_tablerows(*icas_row: np.ndarray[*Ica*], pf_cells: np.ndarray[TableCell]*) → Iterator[TableRow]

property panflute_tablecells: np.ndarray[TableCell]

short_caption: Optional[ListContainer[Inline]] = None

to_panflute_ast() → panflute.table_elements.Table

to_pantablemarkdown() → *pantable.ast.PanTableMarkdown*
return a PanTableMarkdown representation of self

to_pantablestr() → *pantable.ast.PanTableStr*
return a PanTableStr representation of self

All contents are stringified so it is lossy.

class pantable.ast.**PanTableAbstract**(*cells: Union[TableArray, np.ndarray[Union[ListContainer, str]]], caption: Union[ListContainer[Block], str], icas_rowblock: np.ndarray, icas_row: np.ndarray, icas: np.ndarray, short_caption: Optional[Union[ListContainer[Inline], str]] = None, ica_table: Ica = <factory>, spec: Optional[Spec] = None, aligns: Optional[Align] = None, ms: Optional[np.ndarray[np.int64]] = None, ns_head: Optional[np.ndarray[np.int64]] = None*)

Bases: object

an abstract class of PanTables

aligns: Optional[Align] = None

property body_idxs_row: np.ndarray[np.int64]
calculate the i-th body that each row belongs to

negative values means the row is not in a body

caption: Union[ListContainer[Block], str]

cells: Union[TableArray, np.ndarray[Union[ListContainer, str]]]

property contents: np.ndarray[Union[ListContainer, str]]

classmethod default(*shape: Tuple[int, int]*, *has_geometries=False*)
return a default object given shape, etc

This won't work in PanTableAbstract itself but all derived classes including PanTableStr, PanTableMarkdown, PanTableText

property ica_foot: *pantable.ast.Ica*

property ica_head: *pantable.ast.Ica*

ica_table: *Ica*

icas: np.ndarray

property icas_body: np.ndarray[*Ica*]

icas_row: np.ndarray

icas_rowblock: np.ndarray

property icas_rowblock_idxs_row: np.ndarray[np.int64]
calculate the i-th row-block attrs that each row belongs to

property is_body_bodies: np.ndarray[np.bool_]

property is_body_heads: np.ndarray[np.bool_]

property is_foots: np.ndarray[np.bool_]

property is_heads: np.ndarray[np.bool_]

iter_rowblocks(*array: numpy.ndarray*) → List[numpy.ndarray]
break array into list of head, bodies, foot

assume array is iterables of rows

property last_row_of_rowblock_idxs: Set[np.int64]
return a set of the indices of the last row per row-block excluding foot

property m: int

property m_bodies: int

property m_icas_rowblock: int
only one ica per body

property m_rowblocks: int
2 rowblocks per body

property ms: np.ndarray[np.int64]
setter and getter of ms

quirks of dataclass with property see https://stackoverflow.com/a/61480946/5769446

property n: int

ns_head: Optional[np.ndarray[np.int64]] = None

property rowblock_idxs_row: np.ndarray[np.int64]
reverse lookup the index of rowblocks per row

property rowblock_splitting_idxs: np.ndarray[np.int64]
applying np.split(array_of_rows, rowblock_splitting_idxs) would break it back into list of head, bodies, foot

property shape: Tuple[int, int]

short_caption: Optional[Union[ListContainer[Inline], str]] = None

spec: Optional[*Spec*] = None

class pantable.ast.**PanTableMarkdown**(*cells: Union[TableArray, np.ndarray[Union[ListContainer, str]]], caption: str = '', icas_rowblock: Optional[np.ndarray[np.str_]] = None, icas_row: Optional[np.ndarray[np.str_]] = None, icas: Optional[np.ndarray[np.str_]] = None, short_caption: Optional[str] = None, ica_table: Ica = <factory>, spec: Optional[Spec] = None, aligns: Optional[Align] = None, ms: Optional[np.ndarray[np.int64]] = None, ns_head: Optional[np.ndarray[np.int64]] = None, table_width: Optional[float] = None*)

Bases: *pantable.ast.PanTableStr*

similar to PanTableStr, but with all str assumed to be in markdown

to_pancodeblock(*format: str = 'csv', fancy_table: bool = False, include: str = '', csv_kwargs: Optional[dict] = None*) → *pantable.ast.PanCodeBlock*

to PanCodeBlock object

This is lossy as there's no way to encode the geometries of *self.cells* in PanCodeBlock. Use PanTableMarkdown instead if you want to preserve that info.

to_pantable() → *pantable.ast.PanTable*

return a PanTable representation of self

to_str_array(*fancy_table: bool = False*) → np.ndarray[np.str_]

construct a table with both content and ica together

class pantable.ast.**PanTableOption**(*short_caption: str = '', caption: str = '', alignment: str = '', alignment_cells: str = '', width: typing.Optional[typing.List[typing.Union[float, str]]] = None, table_width: typing.Optional[float] = None, header: bool = True, ms: typing.Optional[typing.List[int]] = None, ns_head: typing.Optional[typing.List[int]] = None, markdown: bool = False, fancy_table: bool = False, include: str = '', include_encoding: str = '', format: str = 'csv', csv_kwargs: dict = <factory>*)

Bases: object

options in CodeBlock table

remember that the keys in YAML sometimes uses hyphen/underscore and here uses underscore

alignment: str = ''

alignment_cells: str = ''

caption: str = ''

csv_kwargs: dict

fancy_table: bool = False

format: str = 'csv'

classmethod from_kwargs(***kwargs*) → *pantable.ast.PanTableOption*

header: bool = True

include: str = ''

include_encoding: str = ''

property kwargs: dict
to dict without the defaults

expect *self.from_kwargs(**self.kwargs) == self*

markdown: bool = False

ms: Optional[List[int]] = None

normalize(*shape: Tuple[int, int]*)
assume the types are correct. Normalize what's beyond type-correctness.

e.g. from PanCodeBlock to PanTableStr should uses this

ns_head: Optional[List[int]] = None

short_caption: str = ''

simplify()
Reduced equivalent attrs to simplest form

e.g. from PanTableStr to PanCodeBlock should uses this

table_width: Optional[float] = None

to_spec(*size: int*) → *pantable.ast.Spec*
to Spec

assume normalized self.

width: Optional[List[Union[float, str]]] = None

class pantable.ast.**PanTableStr**(*cells: Union[TableArray, np.ndarray[Union[ListContainer, str]]], caption: str = '', icas_rowblock: Optional[np.ndarray[np.str_]] = None, icas_row: Optional[np.ndarray[np.str_]] = None, icas: Optional[np.ndarray[np.str_]] = None, short_caption: Optional[str] = None, ica_table: Ica = <factory>, spec: Optional[Spec] = None, aligns: Optional[Align] = None, ms: Optional[np.ndarray[np.int64]] = None, ns_head: Optional[np.ndarray[np.int64]] = None, table_width: Optional[float] = None*)

Bases: *pantable.ast.PanTableAbstract*

similar to PanTable, but with panflute ASTs as str

TableArray should have content type as str although not strictly enforced here

TODO: check that icas* are always empty and remove them

TODO: implement auto_width

auto_width(*override_width: bool = False, cell_width_func: Optional[Callable[[str], int]] = <function cell_width_func>*)
calculate column widths

assume a normalized table

caption: str = ''

icas: Optional[np.ndarray[np.str_]] = None

icas_row: Optional[np.ndarray[np.str_]] = None

icas_rowblock: Optional[np.ndarray[np.str_]] = None

short_caption: Optional[str] = None

table_width: Optional[float] = None

to_pancodeblock(*format: str = 'csv'*, *include: str = ''*, *csv_kwargs: Optional[dict] = None*) → *pantable.ast.PanCodeBlock*

to PanCodeBlock object

This is lossy as there's no way to encode the geometries of *self.cells* in PanCodeBlock. Use PanTableMarkdown instead if you want to preserve that info.

to_pantable() → *pantable.ast.PanTable*

return a PanTable representation of self

to_pantableoption(*format: str = 'csv'*, *fancy_table: bool = False*, *include: str = ''*, *csv_kwargs: Optional[dict] = None*) → *pantable.ast.PanTableOption*

class pantable.ast.**PanTableText**(*cells: Union[TableArray, np.ndarray[Union[ListContainer, str]]]*, *caption: str = ''*, *short_caption: Optional[str] = None*, *ica_table: Ica = <factory>*, *spec: Optional[Spec] = None*, *aligns: Optional[Align] = None*, *ms: Optional[np.ndarray[np.int64]] = None*, *ns_head: Optional[np.ndarray[np.int64]] = None*, *table_width: Optional[float] = None*)

Bases: *pantable.ast.PanTableStr*

a quick and dirty PanTableStr without Ica

Except for ica_table, If you try to access icas* and any methods that use them, it will errs.

icas: ClassVar = None

icas_row: ClassVar = None

icas_rowblock: ClassVar = None

class pantable.ast.**Spec**(*aligns:* Align, *col_widths: Optional[np.ndarray[np.float64]] = None*)

Bases: object

a class of spec of PanTable

aligns: *Align*

col_widths: Optional[np.ndarray[np.float64]] = None

classmethod default(*n_col: int = 1*) → *pantable.ast.Spec*

classmethod from_panflute_ast(*table: panflute.table_elements.Table*) → *pantable.ast.Spec*

property size: int

to_panflute_ast() → List[Tuple]

class pantable.ast.**TableArray**(*contents: 'np.ndarray[Union[ListContainer, str]]'*, *geometries: 'Optional[np.ndarray[np.int64]]' = None*)

Bases: object

property cannonical: *pantable.ast.TableArray*

return a cell array where spanned cells appeared in cannonical location only

top-left corner of the grid is the cannonical location of a spanned cell

contents: np.ndarray[Union[ListContainer, str]]

classmethod default(*shape: Tuple[int, int]*, *has_geometries=False*) → *TableArray*

geometries: Optional[np.ndarray[np.int64]] = None

is_at(*i: int*, *j: int*) → bool

is_block(*i: int*, *j: int*) → bool

put(*content: Union[panflute.containers.ListContainer, str]*, *row_span: int*, *col_span: int*, *i: int*, *j: int*, *overwrite: bool = False*)
put content in self

property shape: Tuple[int, int]

shape_at(*i: int*, *j: int*) → Tuple[int, int]

stringified(*width: int = 15*, *cannonical=True*) → *pantable.ast.TableArray*
return stringified TableArray

Parameters **width** (*int*) – width per column

pantable.ast.**cell_width_func**(*string: str*, *offset: int = 3*) → int
return max no. of characters +3 among lines in the cell

The +3 match the way pandoc handle width, see jgm/pandoc commit 0dfceda

pantable.ast.**single_para_to_plain**(*elem: panflute.containers.ListContainer*) → panflute.containers.ListContainer
convert single element to Plain

if *elem* is a ListContainer of a single Para, then convert it to a ListContainer of Plain and return that. Else return *elem*.

pantable.codeblock_to_table module

pantable.codeblock_to_table.**codeblock_to_table**(*options: Optional[dict] = None*, *data: str = ''*, *element: Optional[CodeBlock] = None*, *doc: Optional[Doc] = None*) → Union[Table, list, None]

pantable.io module

pantable.io.**dump_csv**(*data: np.ndarray[np.str_]*, *options:* PanTableOption) → str
dump data as CSV string

pantable.io.**dump_csv_io**(*data: np.ndarray[np.str_]*, *options:* PanTableOption) → str
dump data as CSV

it will mutate options.include if it is an invalid write path.

pantable.io.**load_csv**(*data: str*, *options:* PanTableOption) → List[List[str]]
loading CSV table

Note that this can emit EmptyTableError, FileNotFoundError

pantable.io.**load_csv_array**(*data: str*, *options:* PanTableOption) → np.ndarray[np.str_]
loading CSV table in *numpy.ndarray*

Note that this can emit EmptyTableError, FileNotFoundError

pantable.table_to_codeblock module

pantable.table_to_codeblock.**table_to_codeblock**(*element: Optional[Table] = None*, *doc: Optional[Doc] = None*, *format: str = 'csv'*, *fancy_table: bool = False*, *include: str = ''*, *csv_kwargs: Optional[dict] = None*) → Optional[*PanTable*]

convert Table element and to csv table in code-block with class "table" in panflute AST

pantable.util module

exception pantable.util.**EmptyTableError**

Bases: Exception

pantable.util.**convert_texts**(*texts: Iterable*, *input_format: str = 'markdown'*, *output_format: str = 'panflute'*, *standalone: bool = False*, *extra_args: Optional[List[str]] = None*) → List[list]

run convert_text on list of text

pantable.util.**convert_texts_fast**(*texts: Iterable*, *input_format: str = 'markdown'*, *output_format: str = 'panflute'*, *extra_args: Optional[List[str]] = None*) → List[list]

a faster, specialized convert_texts

should have identical result from convert_texts

pantable.util.**eq_panflute_elem**(*elem1: Element*, *elem2: Element*) → bool

pantable.util.**eq_panflute_elems**(*elems1: List[Element]*, *elems2: List[Element]*) → bool

pantable.util.**get_types**(*cls: Any*) → Dict[str, tuple]

returns all type hints in a Union

c.f. https://stackoverflow.com/a/50622643

pantable.util.**get_yaml_dumper**()

pantable.util.**iter_convert_texts_markdown_to_panflute**(*texts: Iterable[str]*, *extra_args: Optional[List[str]] = None*) → Iterator[ListContainer]

a faster, specialized convert_texts

pantable.util.**iter_convert_texts_panflute_to_markdown**(*elems: Iterable[ListContainer]*, *extra_args: Optional[List[str]] = None*, *seperator: str = 'FMZVSXSPGQJOTTSMZIGTKWWZFTTCP-GYSEFASWQYZUZMHMBTLZLRO-PHYXISE-CEFWMCOMEFRXOVBNXIBTVIUXHAYZOYGFPRJVKLSMZI*) → Iterator[str]

a faster, specialized convert_texts

Parameters

- **elems** (*list*) – must be list of ListContainer of Block. This is more restrictive than convert_texts which can also accept list of Block
- **seperator** (*str*) – a string for seperator in the temporary markdown output

pantable.util.**parse_markdown_codeblock**(*text: str*) → dict

parse markdown CodeBlock just as *panflute.yaml_filter* would

useful for development to obtain the objects that the filter would see after passed to *panflute.yaml_filter*

Parameters **text** (*str*) – must be a single codeblock of class table in markdown

CHAPTER

SEVEN

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

p

INDEX

R

S

T

W